

Task 1

```
rishill10@ASUS16X: ~/Week4 × + ▾
rishill10@ASUS16X:~$ mkdir Week4
rishill10@ASUS16X:~$ cd Week4
rishill10@ASUS16X:~/Week4$ nano task1.c
rishill10@ASUS16X:~/Week4$ gcc task1.c -o task1
rishill10@ASUS16X:~/Week4$ ./task1
Parent Process Started
Parent PID: 728
Child 1 started. PID = 729 Parent PID = 728
Child 2 started. PID = 730 Parent PID = 728
All 3 child processes have been created.

Using wait():
Child 3 started. PID = 731 Parent PID = 728
Child 1 completed. PID = 729
wait() detected child PID 729 finished with exit status 10

Using waitpid():
waitpid() detected child PID 729 finished with exit status 10
Child 2 completed. PID = 730
waitpid() detected child PID 730 finished with exit status 11
Child 3 completed. PID = 731
waitpid() detected child PID 731 finished with exit status 12
Parent process completed.
rishill10@ASUS16X:~/Week4$ |
```

```
rishill10@ASUS16X: ~/Week4 × + v
GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed. PID = %d\n",
                i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("\nUsing wait():\n");

    pid_t finished = wait(&status);

    printf("wait() detected child PID %d finished with exit status %d\n",
        finished, WEXITSTATUS(status));

    printf("\nUsing waitpid():\n");
```

```
    for (int i = 0; i < 3; i++)
    {
        waitpid(pid[i], &status, 0);

        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid[i], WEXITSTATUS(status));
    }

    printf("Parent process completed.\n");

    return 0;
}
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Task 2

```
rishill10@ASUS16X: ~/Week4  ×  Ubuntu  ×  +  ▾  
rishill10@ASUS16X:~$ cd Week4  
rishill10@ASUS16X:~/Week4$ nano task2.c  
rishill10@ASUS16X:~/Week4$ gcc task2.c -o task2  
rishill10@ASUS16X:~/Week4$ ./task2  
Parent process: PID = 751  
Child PID = 752  
Parent sleeping...  
Child process: PID = 752  
Child process terminating...  
Parent terminating...  
rishill10@ASUS16X:~/Week4$ ./task2  
Parent process: PID = 820  
Child PID = 821  
Parent sleeping...  
Child process: PID = 821  
Child process terminating...
```

```
rishill10@ASUS16X: ~/Week4  ×  Ubuntu  ×  +  ▾
GNU nano 8.7.1 task2.c *
\#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");
        sleep(30);
        printf("Parent terminating...\n");
    }

    return 0;
}
```

```
rishill10@ASUS16X: ~/Week4  ×  Ubuntu  ×  +  ▾
rishill10@ASUS16X:~$ ps -el | grep task2
rishill10@ASUS16X:~$ ps -el | grep task2
0 S  1000      820      525 0  80  0 -  692 hrtime pts/0    00:00:00 task2
1 Z  1000      821      820 0  80  0 -    0 -      pts/0    00:00:00 task2
rishill10@ASUS16X:~$
```

```
rishill10@ASUS16X: ~/Week4  ×  Ubuntu  task2.c
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    pid = fork();
    if (pid == 0)
    {
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("Zombie process removed.\n");
    }
    return 0;
}
```

```
rishill10@ASUS16X: ~/Week4  ×  Ubuntu  +  ▾
rishill10@ASUS16X:~$ cd Week4
rishill10@ASUS16X:~/Week4$ nano task2.c
rishill10@ASUS16X:~/Week4$ gcc task2.c -o task2
rishill10@ASUS16X:~/Week4$ ./task2
Parent process: PID = 751
Child PID = 752
Parent sleeping...
Child process: PID = 752
Child process terminating...
Parent terminating...
rishill10@ASUS16X:~/Week4$ ./task2
Parent process: PID = 820
Child PID = 821
Parent sleeping...
Child process: PID = 821
Child process terminating...
Parent terminating...
rishill10@ASUS16X:~/Week4$ nano task2.c
rishill10@ASUS16X:~/Week4$ nano task2.c
rishill10@ASUS16X:~/Week4$ gcc task2.c -o task2
rishill10@ASUS16X:~/Week4$ ./task2
Child process terminating...
Zombie process removed.
rishill10@ASUS16X:~/Week4$ |
```

